Testing the Test Oracle: A Harness for Meaning Preservation in LLM-Expanded AAC

Liam Bray
Independent Researcher
liambray_@outlook.com

Nicholas Cheng
Independent Researcher
nicholasicheng6@gmail.com

***Abstract*—When a language model expands a few AAC keywords into a sentence, `supervisor read` can lose a "not". Catching that needs a test oracle, an automated checker that is itself software and can be wrong. We built a test harness and tested its oracle by mutation. Two defects in our harness, each traced to a regression test, show why: relaxing a rule to remove a false alarm made the oracle accept a changed sentence (the relaxed rule detected only 4/6 acceptance mutants), and an input-builder bug showed models chips that contradicted the source in 7 of 139 inputs. On real model outputs the oracle caught 193/196 seeded mutants with 0/100 false alarms, against 138/196 for string overlap with 16/100 false alarms; a near-circular surface lexicon matched its detection but not its false alarms. Its labels agree with human coders ($\kappa = 0.97$, 0.94 on two output sets). On 547 human-written pairs from 20 writers, four models read bare keywords as affirmative in 99.8% of outputs, and a prompt legend does not change this. A speech-act chip raised the intended speech act by 35.3 pp, positive for every writer, yet kept it in only 73.24% of targets on the confirmation set. No AAC users took part.**

***Index Terms*—accessibility, AAC, LLM testing, test oracle, requirements, regression testing**

I. Introduction

Some people who use augmentative and alternative communication (AAC) compose a message from a few keywords, because each selection costs time and effort [1], [2]. A language model then expands the keywords into a sentence [3], [2]. Typing `supervisor read` might mean "The supervisor read" or "The supervisor did not read" (Fig. 1). Whether the model erred depends on what the user supplied, and a text-similarity score cannot know that. Symbol- and grid-based AAC is out of scope.

This paper asks which output changes count as failures, given exactly what the user supplied, and how to test the checker that decides. The software-engineering problem is the oracle. Deciding whether an output kept the meaning needs a checker, here an LLM that extracts a few explicit labels. An unvalidated checker silently decides every result, so we treat testing the oracle as a first-class contribution beside testing the models. We contribute: (1) a harness with five requirements R1–R5 that have stated sources and executable tests (§II); (2) tests of the oracle, a fault-by-test matrix, and two defects we made ourselves, each traced from symptom to regression test (§IV); (3) evidence on human-written pairs and on a machine-written confirmation set that the tests find real, model-specific behavior (§III); and (4) the harness and stored outputs as an artifact. The harness is for developers, not for assessing AAC users or their messages.

bare `Keywords: supervisor read` ⇒ "The supervisor read." (affirmative)
+chip `Keywords: [NOT] supervisor read` ⇒ "The supervisor did not read." (negated)
full `The user typed: The supervisor does not read my schedule.` ⇒ "The supervisor does not read my schedule." (negated)
hidden source: "The supervisor does not read my schedule."

Assumption vs. Violation. The user typed no word for "not", so the affirmative output is an *Assumption*, charged to the input design and not to the model. If the user supplies `[NOT]` and the output drops it, that is a *Violation*, charged to the model.

Fig. 1. A stored output on a human-written pair (Gemma-3-4B); each input is the exact string sent to the model. The example is the first match in SHA-256 order of family ID with a wrong bare output and a right chip output, so it favours failures and is not typical. For this binary dimension a bare output is wrong in about half of cases by design (49.8% [49.5, 50.0]%, 7 writers), which is not a finding; with the chip the output is right in 99.3% [98.6, 99.8]%. The charging rule is our design choice, not a result.

II. Requirements and Harness

Families and channels: A *family* is a minimal pair of source messages that share content words and differ on one *critical* dimension: polarity, agent (who acts), certainty, speech act (question, request or statement), politeness, or time/quantity. Each variant is rendered through five *channels*, which say how much of the message the user supplied: *bare* keywords; *+word*, which adds one noncritical content word; *+chip*, which adds the critical value as a chip such as `[NOT]`; *+all*, with every chip the variant needs; and *full*, the source sentence itself. Every variant needs at least one chip, so the unmarked reading gets a counter-chip (e.g., `[PLAIN]`). A chip is a *probe* of whether supplied information is used, not a proposed interface: its real cost depends on the access method (a dwell, a menu traversal, a grid button), and it adds a vocabulary to learn and an error-recovery path.

Requirements: For output y from input x, let $L_d(y)$ be the extracted value on dimension d and $C(x)$ the commitments that x carries. Table I states each requirement with its source and its test. A *Violation* means a supplied value was not kept (R1, R2); an *Assumption* means the output committed to a value the input left open (R3). A reference-fidelity score would charge the model for what the interface omitted, and a similarity score would pass an added "please" on a typed sentence. All five requirements are our proposal, derived from

the structure of the task and from defects we met; none was derived with AAC users or clinicians. The declared policy behind R2, do not change typed wording unless the person asks for rewriting, protects authorship. Some users prefer that a system rewrite their text, so R2 is a policy to test, not a recommendation.

Harness: Fig. 2 shows the pipeline. A family specification is rendered into channels, and the R5 validator checks that each displayed chip matches the claim. A generator produces outputs. An extractor (gpt-6.1-sol, a different model from those under test) answers the family's critical question and five general questions from the output alone; added-content extraction also sees the visible input. Two extractor prompts (J1, J2) differ in wording and option order, and critical disagreements go to a cross-family arbiter (claude-opus-5-5) or stay unknown. Verdicts are then computed in code from labels, item specification and channel, so a rule change re-scores every stored output without model calls. The oracle decides only whether a fixed set of explicit, text-level values is kept. It does not judge fluency, referents or whether anyone would accept the sentence.

Adding a chip: worked example and cost: Suppose a developer wants a `[THANKS]` chip for the politeness dimension. The chip table in `src/labels.py` gets 1 new entry:

```
"[THANKS]": ("politeness",
frozenset({"thanks/apology"}))
```

The verdict code (`src/attribute.py`, 267 lines) and the R5 validator (`src/validate_cue_inputs.py`, 333 lines) both read that table, so neither changes. In a dry run (`extend_cost.py`) the validator then marks `Keywords: [THANKS] extra pillow` compatible with a thanks/apology source and conflicting with a plain one, and re-scoring all 22,960 stored human-written-pair output rows took 3.0 s with 0 model calls. A new dimension, not just a new chip, also needs a codebook option and an extractor question, which means new extractor calls; we did not measure that cost. A codebook change works the same way: recoding "Can you…?" from request to question re-scored 3,003 stored confirmation-set rows with no model calls (§III), because verdicts are computed in code. No developer other than the authors has used the harness. Regenerating outputs needs the API models, whose versions may be retired and whose outputs are not bit-reproducible.

Evidence sets: All items are authored or crowd-written, not messages from AAC users. The *human-written pairs* (primary) are 547 pairs kept of 574 candidates, written by 20 non-author writers over six dimensions and kept after three blind independent checks; 546 families are analysed. The *confirmation set* has 238 fresh machine-written families on three dimensions, whose three contrasts were pre-specified in a git-timestamped plan frozen before generation. The *development set* has 60 authored families used to develop prompts and codebooks. *Corpus transfer* uses crowd-written fictional messages from AACText [4], and the *composition study* (R4) adds a second chip to 80 scenarios. Four models run with one fixed one-sentence prompt (greedy): Qwen2.5-7B-Instruct, Llama-3.1-8B-Instruct and Gemma-3-4B-it, three open models run locally with 4-bit quantization, and one API model, gpt-6.1-terra. All intervals are pointwise 95% percentile intervals from writer-cluster (human-written pairs) or seed/construction-cluster (confirmation set) bootstraps, with 10,000 resamples over a fixed model roster.

III. Results: Does the Harness Find Real Behavior?

A. Primary replication on human-written pairs

Table II gives three contrasts. *Act-chip gain* is the speech-act chip minus a length-matched word. *Net politeness addition* is politeness added to plain typed sentences minus politeness removed from polite ones. *Affirmative default* is the share of bare polarity outputs that are affirmative. The plan called them P1–P3 (unrelated to prompts). They were pre-specified for the confirmation set; on the human-written pairs they are an exploratory replication and were not preregistered. Each rests on only 6–7 writers, so a percentile bootstrap cannot give a p below its resampling floor and its intervals are probably too narrow. We therefore recomputed inference at the writer level. An exact sign-flip randomization over writers gives one-sided $p = .0156$, .0156 and .0078 (Holm .031, .031, .023), the smallest values possible with so few writers, and every writer's mean is positive in all three contrasts. The t intervals are wider than the percentile ones (Table II). The effects are heterogeneous across models for the act chip (Llama +18.7 pp, gpt-6.1-terra +50.7) and for politeness (Gemma +22.5, Llama +1.1; permutation $p = .0001$ for each). We claim direction and consistency, not interval width. The chip gain is positive on all six dimensions, between 35.3 and 82.7 pp, with every lower bound above 31.2 pp. Collapsing near-duplicate pairs, excluding one writer, reversing the arbiter's judgment calls or using all candidates flips no decision; the largest shift is 1.7 pp (4.4 pp with all candidates). Because the chip text enters the model's prompt, Act-chip gain and Affirmative default are close to manipulation checks of whether the harness separates supplied from missing information, which is their purpose. They bound what a chip can do rather than show how often systems fail. The politeness figures use two readings, named once: *added* is the share of plain sources that gained a marker, *removed* is the share of polite sources that lost one, and *net* is added minus removed.

B. Confirmation set

All three contrasts have the same sign on the confirmation set, and every interval excludes zero (Table II). Act-chip gain and Affirmative default are of similar size to the human-written pairs. Net politeness addition is small, with a lower bound of 2.50 pp. Confirmation is qualified. First, 48 cached requests predate the final item freeze. Second, Act-chip gain relies on post-hoc arbitration of extractor disagreements (259 rows); with those labels masked as unknown it is 31.65 pp, a sensitivity analysis in the last row of Table II. Net politeness addition

TABLE I

TRACEABILITY OF REQUIREMENTS R1–R5: WHERE EACH COMES FROM, WHICH TEST CHECKS IT, AND WHAT THAT TEST HAS FOUND. ALL CHECKS RUN ON STORED OUTPUTS AND RE-SCORE WITHOUT MODEL CALLS. R1–R4 TEST THE SYSTEM UNDER TEST; R5 TESTS THE INPUT BUILDER.

Requirement	Source	Executable test	What the test has found
R1 supplied chip: if $C(x)$ fixes d to c, $L_d(y)$ must be admissible for c	Interface contract: a chip the person supplied is information the person paid for	Chip check on *+chip*/*+all* outputs; Violation if the value differs	Speech-act chips kept in 73.24% of confirmation-set targets vs. 99.20% for the typed sentence; 58 of 208 `[?]` targets not kept
R2 full input: each scored explicit value of a typed sentence is kept, including the absence of a politeness marker	Declared policy (above); exact copy satisfies it	Full-input check on *full* outputs	Politeness added to plain typed sentences: 7.9% (human-written pairs), 8.12% (confirmation set)
R3 ambiguity: if $C(x)$ leaves d open, $L_d(y)$ is not pass/fail; log an Assumption and report its direction	Bare keywords under-specify; a systematic default decides which reading is free	Default report on *bare*/*+word* outputs; 50% paired match is a design ceiling	Affirmative reading on 99.8% (human-written pairs) and 99.37% (confirmation set) of bare polarity outputs
R4 composition: adding a compatible chip for e must not break a kept value on $d \neq e$	Chips are supplied together in practice	Composition check: pass→fail transitions of the protected value, minus the repeat-draw floor	Composition study (§III-D): Qwen loses the agent under `[TELL]` (13.12 pp), Llama loses polarity under `[STATEMENT]` (8.59 pp)
R5 input builder: a test input that claims to supply c must display a chip for c	Defect B below: failures cannot be attributed to a model if the input lied	Validator on the rendered input (`validate_cue_inputs`), run before generation	7/139 restorations contradicted their source; the validator rejects 7/7

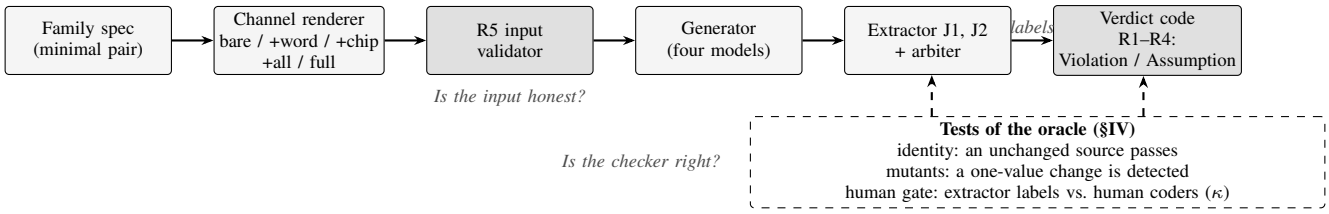


Fig. 2. How a test runs. A message pair is shown to the model with more or less of the information (bare to full). The validator first checks that what we show matches the source (R5), the extractor reads each output into labels, and code, not the LLM, decides Violation or Assumption (R1–R4). The dashed box tests the checker: an unchanged sentence must pass, a one-value change must be caught, and the labels must match human coders. Because verdicts come from stored labels, changing a rule re-scores every output without calling a model.

and Affirmative default contain no arbitrated rows. "Pre-specified and git-timestamped" means the plan was committed to a private repository before generation, which is not independently verifiable. A hash of the protocol was deposited publicly as GitHub release `protocol-hash-2026-11-01`; it post-dates the confirmation-set freeze and so does not certify it.

C. Assumptions, full-input violations and model heterogeneity

Table III separates the two kinds of evidence. In the italic cells the input is ambiguous, so the match is the model's default and is bounded by a 50% design ceiling; it is not accuracy. In the other cells the chip or sentence is visible, so the match measures preservation. Under bare input, 317/319 directional polarity outputs on the confirmation set were affirmative; these are Assumptions, not failures, and the interface decides who pays an extra selection to say "not". That default may be an artifact of heavy compression, so we asked humans: 20 annotators typed keywords for 600 crowd-written AACText messages (1,800 sets). On the 122 messages with a negation cue they dropped it in 20.8% of sets, against 100.0% for a deliberately crude four-word rule-based compressor. So the Assumption channel is realistic but the compressor is not a user model. The affirmative default is also the better guess: among human keyword strings with no negation word, only 2.7% [1.8, 3.7]% came from a negated message, and the models answered affirmatively for 99.9% of them. It is correct on average, but its errors are costly and silent: about 24 negated messages per 1,000 are reversed without any signal, against 0.8 spurious negations. No default removes that exposure, which is why the harness needs a requirement (R3) that logs the Assumption and a test that exposes its direction. A pre-declared three-arm test shows the default is not a prompt-legend artifact: on bare polarity input the affirmative share among committed outputs was 99.5% with the legend, 98.4% with none and 99.9% when the prompt also said markers may be missing and not to assume, across 4 models and both the confirmation set and the human-written pairs (a second draw: 99.3%, 98.6%, 99.9%). Models almost never answered "unclear" (at most 0.7% of outputs in any arm), so a prompt

TABLE II

THE THREE CONTRASTS ON HUMAN-WRITTEN PAIRS (546 FAMILIES; WRITERS PER CONTRAST IN PARENTHESES) AND ON THE CONFIRMATION SET (238 FAMILIES): ONE FIXED PROMPT, FOUR MODELS, 95% CLUSTER-BOOTSTRAP INTERVALS, HOLM-ADJUSTED p OVER THE THREE. HUMAN-WRITTEN PAIRS: t INTERVALS OVER WRITERS (DF = WRITERS -1; PERCENTILE INTERVAL FOR THE DEFAULT, WHOSE t INTERVAL PASSES 100%), AND HOLM p FROM THE EXACT ONE-SIDED WRITER-LEVEL RANDOMIZATION, WHOSE SMALLEST POSSIBLE VALUE WITH 6–7 WRITERS IS $1/2^G$. CONFIRMATION SET: PERCENTILE INTERVALS; $p = .0003$ IS THE SMALLEST HOLM VALUE THAT 10,000 RESAMPLES CAN GIVE. POLITENESS IS NET; ITS ADDED/REMOVED SHARES (%) ARE BELOW IT.

	Human-written	p	Confirmation	p
Act-chip gain, pp	+35.3 [28.9, 41.7] (6)	.031	+31.41 [25.00, 37.94]	.0003
Net politeness, pp	+7.6 [3.7, 11.5] (6)	.031	+6.25 [2.50, 10.00]	.0009
added / removed	7.9 / 0.3		8.12 / 1.88	
Affirmative default, %	99.8 [99.3, 100.0]% (7)	.023	99.37 [98.42, 100.00]%	.0003
Act-chip gain, arbitrated labels masked, pp			+31.65 [25.22, 38.19]	

TABLE III

CONFIRMATION SET: SHARE OF OUTPUTS MATCHING THE INTENDED VALUE, BY CHANNEL (%). ITALIC CELLS (BARE, +WORD) ARE THE MODEL'S DEFAULT UNDER AMBIGUOUS INPUT, BOUNDED BY 50% BY DESIGN FOR THE TWO BINARY DIMENSIONS; THEY ARE NOT ACCURACY. SPEECH ACT IS THREE-VALUED. ONE FIXED PROMPT, FOUR MODELS POOLED; PER-MODEL VALUES AND INTERVALS ARE IN ANALYSIS/R2C/CHANNEL_LADDER.CSV.

Dimension	bare	+word	+chip	+all	full	n per cell
polarity	*49.8*	*49.8*	99.2	96.2	99.8	640
speech act	*39.3*	*41.8*	73.2	74.8	99.2	624
politeness	*50.0*	*50.0*	89.5	87.2	95.0	640

instruction is not a substitute for the requirement and the test.

Question chips are the weakest supplied chip: one chip kept the act in 73.24% of confirmation-set targets, and 42 of the 58 failures were coded as requests ("Can you…?"), a codebook decision. We therefore report both codings of "Can you…?", as a request (stored) and as a question (3,003 confirmation-set rows remapped from stored labels). The confirmation-set Act-chip gain is 31.4 pp as a request and 18.6 pp [10.0, 26.9] as a question; on human-written pairs it is 35.3 pp and 32.5 pp. No Holm decision changes under either coding, and masking each arbiter precedent in turn, or all arbitrated units, moves the three contrasts by at most 0.2 pp. So the size of the Act-chip gain is codebook-dependent, but its sign is not. Full-input violations are model-specific (Table IV): Gemma-3-4B and gpt-6.1-terra added markers, while Llama and Qwen removed markers more often than they added them on the confirmation set. This is a regression to test for per model, not a property of LLMs.

TABLE IV

FULL-INPUT POLITENESS CHANGE BY MODEL. HUMAN-WRITTEN PAIRS: NET ADDITION (PP). CONFIRMATION SET: COUNTS OF PLAIN SOURCES THAT GAINED A MARKER (ADDED) AND POLITE SOURCES THAT LOST ONE (REMOVED), OUT OF 80 FAMILIES EACH.

Model	Human-written, net pp	Added	Removed
Gemma-3-4B	+22.5	20/80	1/80
gpt-6.1-terra	+6.7	5/80	0/80
Llama-3.1-8B	+1.1	1/80	2/80
Qwen2.5-7B	+0.0	0/80	3/80
Pooled	+7.6	26/320	6/320

D. Composition (R4)

The composition study adds a second chip to a single-chip input and counts pass→fail transitions of the first chip's value, minus the disagreement between two greedy repeats of the identical input. It has 80 scenarios, nine inputs each, two encodings of the speech-act chip (`[TELL]` and its alias `[STATEMENT]`), four models and two draws (11,520 requests). Failures depend on both model and encoding. Qwen loses the agent in 84/640 transitions under `[TELL]` (13.12 pp above the repeat floor [10.31, 16.09]) and in 0 under `[STATEMENT]`. Llama loses polarity in 55/640 transitions under `[STATEMENT]` (8.59 pp [6.25, 11.09]) and in 0 under `[TELL]`. The single-chip repeat floor is 0/160 for Llama. An R4 test must therefore fix the encoding, because a failure that appears under one encoding of the same meaning is an encoding defect before it is a model property. This study is exploratory and machine-scored. Critical labels fell back to J1 on 591 of 40,960 scored rows, and the Qwen `[TELL]` examples expose a codebook-rule ambiguity that agreement of the two prompts does not settle.

IV. TESTING THE ORACLE

Every result above is only as good as the checker, so we test it as software, in the spirit of the test-oracle literature [5], [6] and of mutation analysis [7]: an authorized change must pass and an unauthorized one must fail. *Identity test:* an unchanged source injected as its own full-channel output must pass. *Scoped mutants:* a hand-written one-value change to a development-set source must be detected. *Keyed challenge:* hand-written controls and one-feature mutants, with keys hashed before the first extractor call, must be classified as keyed. *Human gate:* extractor labels against human coders.

Two defects, two regression tests: Table V traces the two defects we made while building the harness; neither was seeded. In Defect A, the identity test failed 14 of 2,292 reproductions, all one sentence, "Yes, you can borrow my car this weekend.", coded *statement* against the target *acceptance*. The model had done its job; the checker demanded a distinction the text need not force. The obvious repair, accepting *statement* for an acceptance target, passed identity but failed the next test. The relaxed oracle detected only 4/6 acceptance mutants and missed both statement substitutions ("You borrowed my car this weekend."), whereas without the relaxation it detected

TABLE V
DEFECT-TO-REGRESSION-TEST TRACE.

	A: relaxed acceptance oracle	B: input-builder fallback
Symptom	unchanged source fails (14/2,292)	chip contradicts source (7/139 restorations)
Found by	identity test	audit against R5
Fix	accept *statement* as acceptance	decode rendered input; reject conflicts
Cost of fix	substitutions hidden (4/6)	none; coarser chips stay unknown
Test now in suite	keyed acceptance mutants; quarantine on failure	R5 validator before generation; replay agrees on 139/139

24/24 mutants across groups. Removing a false alarm removed fault sensitivity. Both refusal/acceptance families are therefore quarantined from every primary result (leaving 58 of 60 development families) until a predicate passes both tests. In Defect B, an audit of 139 restored-chip inputs built from AACText messages found 7 whose displayed chip contradicted the stored source, e.g., `[MAYBE]` for a plain assertion, after a silent fallback in our builder. In the 5 `[MAYBE]` messages, all 20 outputs followed the visible chip and failed the source, which a source-fidelity score counts as model failures. The R5 validator decodes only the rendered input and now rejects all 7 before generation. Replayed on every chip input of the confirmation set, the composition study and the human-written pairs (13,496 inputs), it found 0 conflicts; 10,528 were verified compatible and 2,968 (time chips and one long chip alias) are outside its decoder. This covers chip inputs only, not free-text channels. It matches the audit on 139/139 and never certifies the 8 coarser `[SURE]` restorations.

Which test caught which defect: Table VI lists, for each defect class we met, which test caught it, which ran on the affected rule and missed it, and which does not exercise it. No single test catches every class, and the cheapest test (identity) passed the harmful relaxation. It is an inventory of what happened in this project, not a seeded-fault sweep, so it shows that the tests are complementary and not that they find faults we did not anticipate.

Mutation and identity evidence: Table VII lists the evidence, first on authored inputs. The keyed challenge set has four groups of six controls and six one-feature mutants. Its pre-specified gate ($\geq$22/24 mutants detected, $\leq$1/24 false alarms) passes when pooled, with no slack; we read 22/24 as passing a floor, not as evidence of high sensitivity (Wilson interval [74.2, 97.7]%).

Mutation testing on real outputs, with baselines: Authored sentences are easy, so we also seeded one-value mutants into real stored outputs of the human-written pairs (seed fixed in advance; operators re-implemented because the authored mutants were hand-written) and drew 100 surface-edited controls (case, punctuation, contraction; 4 boosters, which the contract tolerates, are excluded from the mutants) from outputs the oracle scored as preserving. Table VII gives the result: the judge caught 193/196 mutants with 0/100 false alarms, matching the authored-sentence result (120/120). Of its 3 misses, two are time shifts that left an explicit past-tense phrase ("in the past") beside the shifted verb, so the mutant is mixed-signal; the third is an equivalent mutant (the source was already a question). A surface lexicon (negation, hedge, politeness, tense and question cues) scores 190/196 with 4/100 false alarms, and string overlap 138/196 with 16/100. The lexicon is near-circular, since it matches the cues the operators insert and delete; its score shows that the operators are lexically shallow, not that a lexicon is an adequate oracle, and the judge's edge shows mainly on controls.

Human gate: The first human gate, with two annotators, was run on the development items (the extraction inventory used to develop the codebook), with plurality references and tied items excluded rather than adjudicated. Its pre-specified criterion ($\kappa \geq 0.6$) passes on all 7 fields. It validates extraction of explicit markers on those items, not repaired text or corpus-transfer readings. Two later checks use outputs the harness had not been developed on: outputs for human-elicited keywords and outputs for human-written pairs (Table VII). The excluded items are 10 of 161 and 2 of 152. Recomputed with the stricter rule that ties and unsure count as disagreement, critical κ on the test split is .930 (development items), .925 [.874, .971] (human-elicited keywords) and .931 [.868, .981] (human-written pairs), so the gate still passes; the weakest field is added content on human-written pairs (.692, lower bound .541). Those checks are the oracle's evidence on the headline-type data, though their coders were also taught the codebook, and the verdict κ values on human-written pairs rest on small counts of positive verdicts. Machine agreement is not validation: on "I can't wait to go to the party.", both extractor prompts, the rule-based extractor and two further machine coders answered *negated*, and both human annotators answered *affirmative*, as keyed.

V. RELATED WORK

Test oracles, metamorphic testing and requirements for ML: Whether an output is correct is the oracle problem [6]; for programs without a practical oracle, partial or pseudo oracles are standard [5], and metamorphic testing checks relations between runs instead [8]. Mutation analysis measures a test suite by whether it detects seeded faults [7]; we apply that idea to the oracle, not the system. Requirements engineering for ML stresses that data-dependent behavior needs explicit, testable requirements [9]; our R1–R5 are one such set, with sources and tests stated.

Testing LLM systems: CheckList organizes behavioral tests into minimum-functionality, invariance and directional tests [10]; Cho et al. catalogue metamorphic relations for LLM-based NLP [11]; COLLIE tests compositional constraints [12]; Sensei combines a user simulator with assertions and metamorphic relations [13]. We claim no new testing technique. LLM judges show position, verbosity and self-preference biases [14], [15], and similarity metrics can miss negation [16]. Pauli et al. show that preservation metrics look valid when test data confound style change with content

TABLE VI

FAULT-BY-TEST MATRIX. *caught*: THE TEST FAILED ON THE DEFECT. *missed*: THE TEST RAN ON THE AFFECTED RULE AND PASSED. –: THE TEST DOES NOT EXERCISE THIS FAULT. COUNTS ARE FROM THE IDENTITY, SCOPED-MUTANT AND KEYED-CHALLENGE RUNS (`ANALYSIS/ORACLE/`), THE R5 REPLAY AND THE TWO HUMAN-AGREEMENT SETS.

Defect class	Identity	Scoped mutants	Keyed challenge	R5 validator	Human gate
Over-strict acceptance rule: a verbatim source fails	caught (14/2,292)	–	missed (0/24 control false alarms)	–	–
Over-relaxed acceptance rule: a statement substitution passes	missed (0/120 false alarms)	missed (120/120 detected, but its acceptance mutants were refusals)	caught (4/6 vs. 24/24 without the relaxation)	–	–
Builder shows a chip that contradicts the source	–	–	–	caught (7/7)	–
Extractor error shared by machine coders ("I can't wait" read as negated)	–	–	caught (round-1 labels match the key in 22/24; the misses include this sentence)	–	caught (both human coders as keyed)
Extractor insensitive to a one-value edit	–	none found (120/120)	none found (24/24, raw rule)	–	–

TABLE VII

ORACLE EVIDENCE. MUTATION AND IDENTITY CHECKS ARE ON AUTHORED SENTENCES OR, IN THE REAL-OUTPUT ROWS, ON RULE-SEEDED EDITS OF STORED OUTPUTS (WILSON 95% INTERVALS); NEITHER IS AN ERROR RATE ON UNEDITED OUTPUTS. HUMAN ROWS COMPARE EXTRACTOR LABELS WITH THE PLURALITY OF TRAINED HUMAN CODERS; ITEMS WHOSE HUMAN LABELS HAVE NO DETERMINATE CRITICAL VALUE (ALL CODERS UNSURE, OR A TIED PLURALITY) ARE EXCLUDED.

Check	Result
Identity: unchanged source, false alarms	0/120
Scoped single-value mutants detected	120/120
Keyed challenge mutants detected (gate)	22/24
Keyed challenge controls, false alarms	0/24
acceptance group, relaxed rule	4/6
all groups, without the relaxation	24/24
Real outputs (196 mutants, 100 controls): mutants caught; false alarms	
judge (J1+J2, arbiter)	193/196; 0/100
surface lexicon (near-circular)	190/196; 4/100
string overlap	138/196; 16/100
Human gate, development items (187 items, two coders): held-out κ, 7 fields	0.91–0.97
Outputs for human-elicited keywords (4 coders, 151 of 161 items): critical κ [CI]	0.97 [0.94, 0.99]
raw agreement, critical label	97.4%
verdict agreement	151/151
Outputs for human-written pairs (3 coders, 150 of 152 items): critical κ [CI]	0.94 [0.89, 0.97]
raw agreement, critical label	94.0%
verdict κ	0.81–0.85

errors [17]; our identity and mutant tests apply that idea to a checker.

Preservation and LLMs in AAC: Qu et al. evaluate speech-act retention in style transfer [18]; rewriting can keep semantic similarity while erasing pragmatic markers [19]; and LLMs favor particular politeness strategies [20]. AAC systems studied with users expose intent controls [1], [21], [22], [23]; Lacosse et al. keep bracketed function words such as "not" [24]; Choudhury et al. test whether affect cues reach the output [25]; R-SPEAK scores faithfulness and was co-designed with people with aphasia [26]. We add a conditional specification of which commitments are testable under each input, and tests of the oracle that checks them. Accessibility engineering spans the life cycle [27], but conformance checks do not see whether generated text keeps a user's meaning.

VI. THREATS TO VALIDITY

Construct. Labels are coarse, explicit categories; "plain" means no explicit marker, and referents, negation scope and certainty degree are only partly covered. Passing the checker is weaker than keeping the typed wording, and a scored repair is not necessarily a sentence someone could say. *Oracle.* All rates depend on an LLM extractor and arbiter; the arbiter ran after outcomes were known, and human coders were taught the codebook. The first human gate's κ is on development items, not on confirmation data. The defect trace has two defects, both found by our own process; it shows that the process works, not that it finds defects we did not anticipate. The surface baseline is near-circular because its lexicon matches the mutation operators, and we ran no NLI baseline. *Internal.* The confirmation set has early cached requests and post-hoc arbitration; the human-written pairs are not preregistered and have few writers per contrast. *External.* Four models, three small and quantized; one prompt; English; constructed and crowd-written, not AAC-user, messages. Model versions may be retired, and API outputs are not bit-reproducible. *Conclusion.* Effects are model-specific, and zero observed events do not establish zero risk. Selection counts omit reading, navigation and fatigue [28].

VII. CONCLUSION

An oracle for meaning preservation is software and needs tests of its own. Ours caught a repair that hid real substitutions and a builder that supplied the wrong chip, and its labels agree with trained human coders on two sets of outputs. With it, human-written pairs show that bare keywords leave a default reading and that some models change typed text. They also show that a supplied speech-act chip helps but

is not always kept. Families, requirements, harness, oracle tests, stored outputs and analysis code are available at https://github.com/1unarzDev/aac_voice_preservation.

Ethics and Positionality

No data were collected from AAC users; AACText is public under CC BY 4.0. All annotators were adult non-authors recruited from the authors' university classmates, identified only by an ID; each consented and was reimbursed for their time (payment and food). They labelled model outputs and audited items under a fixed codebook, blinded to model, condition and source. Some also chose keywords for public AACText messages, and some wrote new sentence pairs, which they released under CC BY 4.0 with recorded consent. No personal or health data were collected. As independent researchers, we did not seek an institutional review determination. No AAC users, speech-language pathologists or autistic advisers took part in this work, which limits it: every requirement here is our assumption about what matters, not a finding about what AAC users want, and the requirements should be revisited with AAC users before any deployment decision. We drew on writing by and with autistic and AAC-using adults [29], [30], [28] and on participatory-research and double-empathy work [31], [32] to avoid encoding a group style or deficit; that reading is not a substitute for participation. We assume no "autistic style", and our claims are limited to commitments explicit in the text.

References

[1] S. Valencia, R. Cave, K. Kallarackal, K. Seaver, M. Terry, and S. K. Kane, ""The less I type, the better": How AI language models can enhance or impede communication for AAC users," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, 2023.

[2] S. Cai, S. Venugopalan, K. Seaver, X. Xiao, K. Tomanek, M. R. Morris, S. Kane, A. Narayanan, R. L. MacDonald, E. Kornman, D. Vance, B. Casey, S. M. Gleason, P. Q. Nelson, and M. P. Brenner, "Using large language models to accelerate communication for eye gaze typing users with ALS," *Nature Communications*, vol. 15, p. 9449, 2024.

[3] J. Shen, B. Yang, J. Dudley, and P. O. Kristensson, "KWickChat: A multi-turn dialogue system for AAC using context-aware sentence generation by bag-of-keywords," in *Proceedings of the 27th International Conference on Intelligent User Interfaces*. ACM, 2022, pp. 853–867.

[4] K. Vertanen and P. O. Kristensson, "The imagination of crowds: Conversational AAC language modeling using crowdsourcing and large data sources," in *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, 2011, pp. 700–711. [Online]. Available: https://aactext.org/imagine/

[5] E. J. Weyuker, "On testing non-testable programs," *The Computer Journal*, vol. 25, no. 4, pp. 465–470, 1982.

[6] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[7] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[8] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 4:1–4:27, 2018.

[9] A. Vogelsang and M. Borg, "Requirements engineering for machine learning: Perspectives from data scientists," in *2019 IEEE 27th International Requirements Engineering Conference Workshops (REW)*, 2019, pp. 245–251.

[10] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 4902–4912.

[11] S. Cho, S. Ruberto, and V. Terragni, "Metamorphic testing of large language models for natural language processing," in *Proceedings of the 41st IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2025. [Online]. Available: https://valerio-terragni.github.io/assets/pdf/cho-icsme-2025.pdf

[12] S. Yao, H. Chen, A. W. Hanjie, R. Yang, and K. Narasimhan, "COLLIE: Systematic construction of constrained text generation tasks," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/a77eadda332b6d4a9ae1e0e4024555f2-Abstract-Conference.html

[13] J. de Lara, A. del Pozzo, E. Guerra, and J. Sánchez Cuadrado, "Automated end-to-end testing for conversational agents," *Journal of Systems and Software*, vol. 233, p. 112685, 2026.

[14] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2306.05685

[15] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[16] P. Koch, M. Aßenmacher, and C. Heumann, "Pre-trained language models evaluating themselves—a comparative study," in *Proceedings of the Third Workshop on Insights from Negative Results in NLP*, 2022, pp. 180–187. [Online]. Available: https://aclanthology.org/2022.insights-1.25/

[17] A. B. Pauli, I. Augenstein, and I. Assent, "Mind the style gap: Meta-evaluation of style and attribute transfer metrics," in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Association for Computational Linguistics, 2025, pp. 21 550–21 564.

[18] R. Qu, L. Ungar, and J. Sedoc, "Conditioning on dialog acts improves empathy style transfer," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 13 254–13 271.

[19] S. K. Navneet, J. Chandra, and Y. Zhang, "When AI writes, whose voice remains? quantifying cultural marker erasure across world English varieties in large language models," in *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems*. ACM, 2026.

[20] H. Zhao and R. D. Hawkins, "Comparing human and LLM politeness strategies in free production," in *Proc. 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2025, pp. 16 188–16 216.

[21] S. Valencia, J. Huynh, E. Y. Jiang, Y. Wu, T. Wan, Z. Zheng, H. Admoni, J. P. Bigham, and A. Pavel, "COMPA: Using conversation context to achieve common ground in AAC," in *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. ACM, 2024.

[22] L. Mao, J. H. Lee, Y. Faroqi-Shah, and S. Valencia, "Design probes for AI-driven AAC: Addressing complex communication needs in aphasia," in *Proceedings of the 2025 ACM Designing Interactive Systems Conference*. ACM, 2025, pp. 404–422.

[23] W. Xiang, Y. Xu, Y. Fang, Z. Teng, Z. Jiang, B. Hu, and J. Yang, "SocializeChat: A GPT-based AAC tool grounded in personal memories to support social communication," 2025.

[24] E. Lacosse, P. Tano, and D. C. McNamee, "Multi-system cognitive steering for generative communication," 2026, arXiv preprint.

[25] S. H. Choudhury, A. Kumar, and L. J. Martin, "Evaluating human-LLM representation alignment: A case study on affective sentence generation for augmentative and alternative communication," in *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics, 2025, pp. 1618–1637.

[26] A.-K. Al-Tamimi, J. Andrews, J. Benfield, C. Sweby, C. Gilmartin, R. Lindley, D. Trusson, M. Dziunka, D. Webster, and K. Radford, "R-SPEAK, an AI-based smartphone app to enhance communication in people with expressive aphasia: Qualitative acceptability and usability study," *Journal of Medical Internet Research*, vol. 28, p. e93518, 2026.

[27] D. M. B. Paiva, A. P. Freire, and R. P. d. M. Fortes, "Accessibility and software engineering processes: a systematic literature review," *Journal of Systems and Software*, vol. 171, p. 110819, 2021. [Online]. Available: https://repositorio.ufla.br/items/10c85d43-321a-492d-8e30-cfea1a6898bb

[28] B. Frisch, W. Wade, D. Gaines, M. Kinsella, B. Peters, T. Broderick, and K. Vertanen, "It's complicated: On the design and evaluation of AI-powered AAC interfaces," 2026, position paper, Speech AI for All workshop at CHI 2026.

[29] A. H. Zisk and E. Dalton, "Augmentative and alternative communication for speaking autistic adults: Overview and recommendations," *Autism in Adulthood*, vol. 1, no. 2, pp. 93–100, 2019.

[30] A. L. Donaldson, endever* corbin, and J. McCoy, ""everyone deserves AAC": Preliminary study of the experiences of speaking autistic adults who use augmentative and alternative communication," *Perspectives of the ASHA Special Interest Groups*, vol. 6, no. 2, pp. 315–326, 2021.

[31] C. Nicolaidis, D. Raymaker, S. K. Kapp, A. Baggs, E. Ashkenazy, K. McDonald, M. Weiner, J. Maslak, M. Hunter, and A. Joyce, "The AASPIRE practice-based guidelines for the inclusion of autistic adults in research as co-researchers and study participants," *Autism*, vol. 23, no. 8, pp. 2007–2019, 2019.

[32] D. E. M. Milton, "On the ontological status of autism: The 'double empathy problem'," *Disability & Society*, vol. 27, no. 6, pp. 883–887, 2012.